CSCI 3060U – Course Project (Winter 2026)
Phase #2: Front End Rapid Prototyping

Group Members:

Mansi Kandoi - 100821953

Sneh Patel - 100950609

Jolly Soni - 100876863

Table of Contents

Purpose	**2**
Inputs & Outputs	**3**
Architecture Overview	**4**
UML Diagram	**5**
Class & Method Table	**8**
How to Run	**12**

Purpose:

The purpose of this project phase #2 is to design and implement a first working version of the **Front End** of the Banking System. The Front End is a console-based application that simulates an ATM-style interface for processing basic banking transactions.

The system reads a stream of transaction commands from standard input and interacts with a Current Bank Accounts File to validate and record user actions. During an active session, the Front End processes transactions such as login, withdrawal, deposit, transfer, paybill, and other administrative operations according to the rules defined in Phase 1.

All accepted transactions are recorded internally during the session. When a logout command is issued, the Front End writes a Bank Account Transaction File containing a fixed-length record of every transaction performed in that session. This output is later used by the Back End for overnight processing.

This phase focuses on establishing a clear architecture, readable design, and maintainable source code that aligns with the requirements and test structure defined in Phase 1.

Inputs & Outputs

Inputs

The Front End receives input in two ways:

- **Current Bank Accounts File:**
  The system reads a Current Bank Accounts File that contains information about all existing bank accounts, including account numbers, account holder names, account status, and balances. This file is loaded after a successful login and is used to validate transactions during the session.
- **(No Input File 2 in this Phase):**
  There is no second input file used by the Front End in Phase 2. The Daily Transaction File is produced as an output only when the user logs out and is intended for use by the Back End in a later phase.

Outputs

The Front End produces the following outputs:

- **Standard Output:**
  Messages such as prompts, confirmations, and error messages are printed to standard output. These messages are captured and checked by the automated test scripts from Phase 1.
- **Bank Account Transaction File (Daily Transaction File):**
  When a logout command is processed, the system writes a Bank Account Transaction File. This file contains a fixed-length record of all transactions that were accepted during the session and ends with an end-of-session record.

Architecture Overview

The Front End is a command-line program that handles one banking session at a time. It starts in a logged-out state and reads transaction commands from standard input.

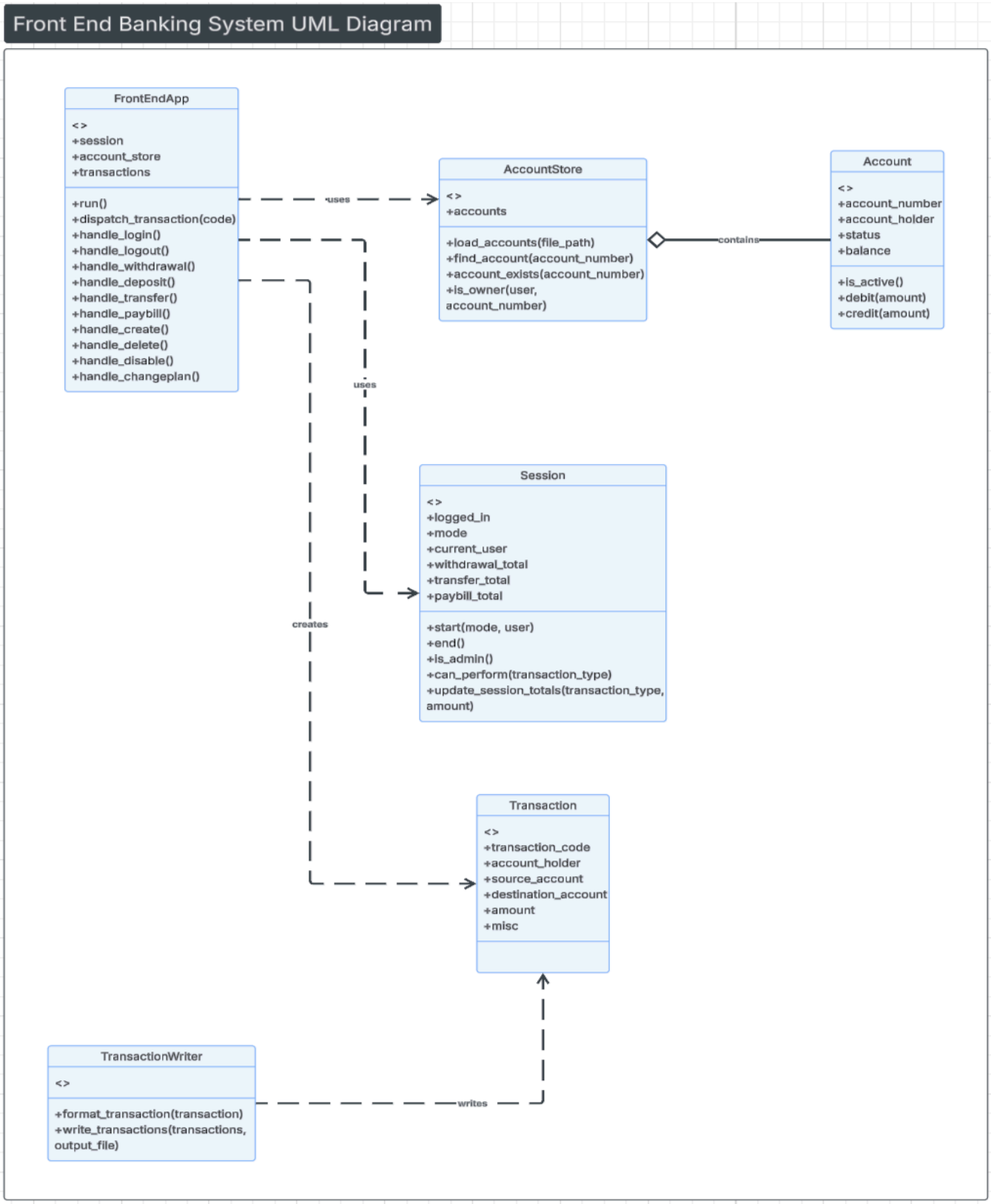
Once a user logs in, the Front End loads the Current Bank Accounts File into memory for validation during the session. For each transaction, the Front End reads the required details, checks the basic rules, and saves it if allowed. What actions are permitted depends on whether the session is in standard or admin mode.

The system does not write transactions to a file right away. Instead, all accepted transactions are stored in memory during the session. This keeps the flow simple and avoids unnecessary file operations.

When the user logs out, the Front End writes all recorded transactions to a Bank Account Transaction File in the required format and adds an end-of-session record. After logging out, no further transactions are accepted until a new login occurs.

The design separates session handling, account data, and transaction processing so the code stays easy to read, easy to update, and easy to test in later phases.

UML DIAGRAM


Front End Banking System UML Diagram
FrontEndApp
<>
+session
+account_store
+transactions
+run()
+dispatch_transaction(code)
+handle_login()
+handle_logout()
+handle_withdrawal()
+handle_deposit()
+handle_transfer()
+handle_paybill()
+handle_create()
+handle_delete()
+handle_disable()
+handle_changeplan()
uses
AccountStore
<>
+accounts
+load_accounts(file_path)
+find_account(account_number)
+account_exists(account_number)
+is_owner(user, account_number)
contains
Account
<>
+account_number
+account_holder
+status
+balance
+is_active()
+debit(amount)
+credit(amount)
uses
Session
<>
+logged_in
+mode
+current_user
+withdrawal_total
+transfer_total
+paybill_total
+start(mode, user)
+end()
+is_admin()
+can_perform(transaction_type)
+update_session_totals(transaction_type, amount)
creates
Transaction
<>
+transaction_code
+account_holder
+source_account
+destination_account
+amount
+misc
TransactionWriter
<>
+format_transaction(transaction)
+write_transactions(transactions, output_file)
writes

Example Walkthrough: One User Session

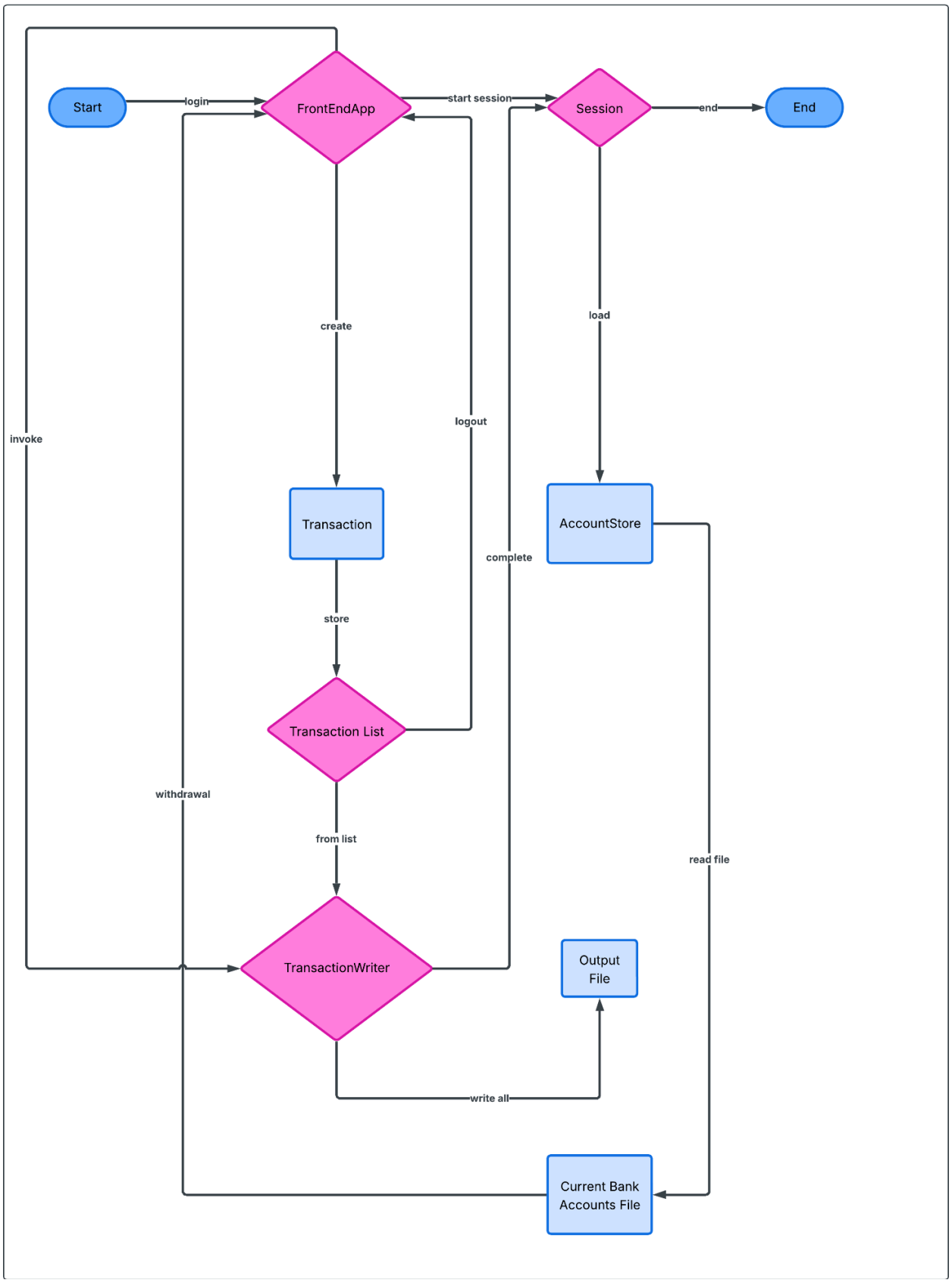
This section explains how the Front End components work together by tracing a single user through the system.

Scenario: A standard user logs in, performs a withdrawal, and then logs out.

1. Login
    - The user enters the login command through standard input.
    - frontend_app.py reads the command and starts a new session using the Session class.
    - The Current Bank Accounts File is loaded by AccountStore, which creates Account objects and stores them in memory.
    - The system is now ready to accept transactions.
2. Withdrawal
    - The user enters the withdrawal command.
    - frontend_app.py checks whether the transaction is allowed using the session state.
    - The account number and amount are read from input.
    - A Transaction object is created to represent the withdrawal and is stored in a list of session transactions.
3. Logout
    - The user enters the logout command.
    - frontend_app.py sends the list of stored Transaction objects to TransactionWriter.
    - TransactionWriter formats each transaction and writes them to the Bank Account Transaction File.
    - The session is ended and the program stops accepting further transactions.

This walkthrough shows how the Front End processes a complete session and how the different components interact without requiring detailed knowledge of the code.

Front End Banking System User Flow



Class & Method Table

FrontEndApp

Class	Method/Field	Intention
FrontEndApp	`session`	Stores the current session state (logged in, mode, user, limits).
FrontEndApp	`account_store`	Holds all loaded accounts so transactions can be validated.
FrontEndApp	`transactions`	Stores accepted transactions during the session until logout.
FrontEndApp	`run()`	Starts the main loop, reads transaction codes from standard input, and keeps the program running.
FrontEndApp	`dispatch_transaction(code)`	Sends each transaction code to the correct handler function.
FrontEndApp	`handle_login()`	Handles login, sets session mode/user, and loads the current accounts file.
FrontEndApp	`handle_logout()`	Ends the session and writes the transaction output file with an end-of-session record.
FrontEndApp	`handle_withdrawal()`	Reads withdrawal details, checks rules, and records the transaction if accepted.
FrontEndApp	`handle_deposit()`	Reads deposit details, checks rules, and records the transaction if accepted.
FrontEndApp	`handle_transfer()`	Reads transfer details, checks rules, and records the transaction if accepted.
FrontEndApp	`handle_paybill()`	Reads paybill details, checks rules, and records the transaction if accepted.
FrontEndApp	`handle_create()`	Admin-only: reads new account info and records a create transaction.
FrontEndApp	`handle_delete()`	Admin-only: reads account info and records a delete transaction.

FrontEndApp	`handle_disable()`	Admin-only: reads account info and records a disable transaction.
FrontEndApp	`handle_changeplan()`	Admin-only: reads account info and records a changeplan transaction.

Session

Class	Method/Field	Intention
Session	`logged_in`	Tracks whether the system is currently in an active session.
Session	`mode`	Stores whether the session is standard or admin.
Session	`current_user`	Stores the account holder name for standard sessions.
Session	`withdrawal_total`	Tracks total withdrawals in this session for standard limit checks.
Session	`transfer_total`	Tracks total transfers in this session for standard limit checks.
Session	`paybill_total`	Tracks total paybills in this session for standard limit checks.
Session	`start(mode, user)`	Starts a session and initializes session state and totals.
Session	`end()`	Ends the session and clears session state.
Session	`is_admin()`	Returns true if the current session is admin mode.
Session	`can_perform(transaction_type)`	Checks whether the transaction type is allowed in the current session state.
Session	`update_session_totals(transaction_type, amount)`	Updates session totals after an accepted transaction.

AccountStore

Class	Method/Field	Intention
AccountStore	`accounts`	Stores all accounts loaded from the current accounts file.
AccountStore	`load_accounts(file_path)`	Reads the current accounts file and builds Account objects in memory.
AccountStore	`find_account(account_number)`	Returns the account object for a given account number, if it exists.
AccountStore	`account_exists(account_number)`	Checks whether an account number exists in the system.
AccountStore	`is_owner(user, account_number)`	Checks whether a given account belongs to the logged-in user (standard mode).

Account

Class	Method/Field	Intention
Account	`account_number`	Stores the account number.
Account	`account_holder`	Stores the account holder name.
Account	`status`	Stores whether the account is active or disabled.
Account	`balance`	Stores the account balance for validation during the session.
Account	`is_active()`	Returns true if the account status is active.

Account	`debit(amount)`	Reduces balance in memory when a withdrawal/transfer/paybill is accepted.
Account	`credit(amount)`	Adds balance in memory when needed for validation (if used).

Transaction

Class	Method/Field	Intention
Transaction	transaction_code	Stores the transaction type code used in the output file.
Transaction	account_holder	Stores the name tied to the transaction (user or admin-provided).
Transaction	source_account	Stores the main account number involved in the transaction.
Transaction	destination_account	Stores the destination account number for transfers, if needed.
Transaction	amount	Stores the transaction amount.
Transaction	misc	Stores extra information like bill company code or transfer detail.

TransactionWriter

Class	Method/Field	Intention
TransactionWriter	format_transaction(transaction)	Converts a Transaction into one fixed-length 40-character record line.
TransactionWriter	write_transactions(transactions, output_file)	Writes all transaction records to the output file and appends the end-of-session record.

How to Run

The Front End is a command-line program and is meant to be run using the shell scripts provided in Phase 1.

For each test case, the shell script runs the program once and:

- provides the transaction commands through standard input
- captures all printed messages from standard output
- supplies the required input and output file paths

The program reads the Current Bank Accounts File when a user logs in. All transactions are processed during the session. When a logout command is given, the program writes the Bank Account Transaction File for that session.

This setup allows all test cases to be run automatically and consistently.